Parameter-Efficient Residual Classification with PyramidNet and Weight Averaging

Sheng Tao

NYU Tandon School of Engineering
Deep Learning, Fall 2026
https://github.com/TolsonRyan/cifar10-resnet-architecture-challenge

Abstract

We investigate CIFAR-10 image classification with a residual convolutional network constrained to fewer than five million trainable parameters. Our final model is a locally implemented PyramidNet-110 with ShakeDrop regularization and 4,804,512 parameters. A shared model is trained from random initialization for 400 epochs, followed by two 20-epoch continuations with different random seeds. We average the final branch weights into one inference network and average its class probabilities across 50 translated and horizontally flipped views. The resulting model achieves 97.90% validation accuracy and 87.60% on the competition's final hidden evaluation subset. Comparisons with wide residual networks, sharpness-aware optimization, and knowledge distillation motivate the final training recipe. Short continuation improves the base model, while weight averaging and denser test-time augmentation provide smaller refinements without increasing the deployed parameter count.

Overview

The project requires a modified ResNet trained from scratch using only the permitted CIFAR-10 training images, with at most 5,000,000 trainable parameters. Residual connections allow the model to learn transformations around an identity path rather than repeatedly reconstructing its input representation (He et al. 2016a). Under a parameter budget, increasing width competes with depth and training cost. Our final design uses gradual channel growth rather than abrupt channel doubling at every stage boundary.

We study how to allocate this budget across residual architectures and training strategies. Initial wide-network experiments compare post-activation and preactivation blocks, standard SGD and sharpness-aware minimization, and a matched distillation continuation. These experiments lead to a deeper PyramidNet recipe, which we refine through short continuation, weight averaging, and test-time augmentation. All models are trained using only the permitted images and locally initialized weights.

Methodology

Data and evaluation protocol

The provided data contain 50,000 labeled 32×32 RGB training images and 2,000 unlabeled competition images. We use a stratified split generated with seed 2026: 45,000 training examples and 5,000 validation examples, respectively 4,500 and 500 per class. The final model and both continuation branches use this same training subset. Validation examples are excluded from gradient updates and normalization statistics.

Images are loaded as RGB uint8 arrays, converted to floating point in $[0, 1]$, and normalized using training-only channel means $(0.491514, 0.482315, 0.446814)$ and standard deviations $(0.247051, 0.243480, 0.261510)$. Predictions follow the official sample submission ID order. Checks enforce image shape, channel format, unique IDs, and integer labels from zero through nine.

All comparisons below use the same 5,000 validation images. We report top-1 accuracy and negative log-likelihood (NLL), with checkpoint selection and inference policy stated for each comparison. The final test result comes from the competition's private evaluation subset.

Residual architecture

PyramidNet gradually increases channel width across residual blocks (Han, Kim, and Kim 2017). Our network has a 16-channel 3×3 stem and 54 basic residual blocks, with 18 blocks in each spatial stage. The output width of block i is

$$c_i = \mathrm{round}(16 + 96i/54), \qquad i = 1, \ldots, 54.$$

The resulting stage-end widths are 48, 80, and 112. Each block applies input batch normalization (BN), a 3×3 convolution, BN and ReLU, another 3×3 convolution, output BN, and ShakeDrop before adding the shortcut. There is no post-addition ReLU. The first block in stages two and three downsamples by two. Its shortcut uses average pooling; additional channels are zero-padded. Thus every block retains an explicit residual addition without a learned projection on the shortcut. Final BN, ReLU, global average pooling, and a ten-class linear layer complete the model.

Gradual growth distributes capacity across depth while zero-padding shortcuts avoid extra projection parameters. Compared with a wider, shallower residual network, this design requires more sequential operations and longer training. The experiments below assess this depth–width trade-off together with the associated training recipe.

Base training

The shared base is trained for 400 epochs using SGD with Nesterov momentum 0.9, batch size 128, weight decay

Component	Channels	Resolution
Stem	16	32×32
Stage 1: 18 blocks	48	32×32
Stage 2: 18 blocks	80	16×16
Stage 3: 18 blocks	112	8×8
Pooling + classifier	10	–

Table 1: Final residual architecture. Parameter count, including BN affine parameters and the classifier, is 4,804,512.

10^{-4}, and initial learning rate 0.1. Five warmup epochs precede per-step cosine decay to approximately 10^{-4}. Training uses BF16 on an RTX 4060 Laptop GPU with 8 GB memory; evaluation uses FP32 with TF32 disabled.

Augmentation consists of reflection-padding by four pixels, random 32×32 crops, horizontal flips, RandAugment with two operations and magnitude nine, and Cutout with side length 16 (Cubuk et al. 2020; DeVries and Taylor 2017). RandAugment operation choices are shared within shuffled groups of 32 images; crop, flip, and Cutout locations are sampled per image. Cutout is applied after normalization and fills the masked region with zero. This implementation detail is preserved in the reproduction code. Mixup, CutMix, and label smoothing are not active in the final recipe.

ShakeDrop perturbs the residual branch during base training (Yamada et al. 2019). Block death probabilities increase linearly to 0.5. A batch-level Bernoulli gate and per-image forward and backward coefficients provide the stochastic residual multipliers. At inference the residual is multiplied by its keep probability. An exponential moving average (EMA) with target decay 0.999 tracks floating model tensors, including BN statistics; integer buffers are copied. Its initial decay is bounded by $(1 + u)/(10 + u)$ at update u.

Two continuation branches and averaging

Both branches start from the base model's epoch-400 EMA weights. Seeds 62026 and 62029 determine their subsequent randomness. Each runs for 20 epochs with fresh SGD momentum, learning rate 0.002 decaying to approximately 0.0001, and the same data augmentation and weight decay. During continuation, only ShakeDrop modules use their deterministic inference multiplier; BN remains in training mode. This reduces residual-branch stochasticity without changing the deployed network.

We use the final Raw endpoints, not the highest-scoring intermediate epoch. All floating tensors are averaged with fixed equal weights,

$$\bar{\theta} = (\theta_A + \theta_B)/2.$$

This includes BN running means and variances. Integer buffers are retained from branch B, and no subsequent BN recalibration is applied. This is a two-branch weight average, related to weight-averaging approaches (Izmailov et al. 2018), rather than a prediction ensemble or an implementation of standard stochastic weight averaging. Only one network is deployed. Each branch follows a 420-epoch path; the shared computation is 400 base epochs plus two separate 20-epoch continuations.

Test-time augmentation

We reflection-pad the input by two pixels and use every integer translation in $\{-2, -1, 0, 1, 2\}^2$, with and without a horizontal flip. The 25 translations produce 50 views. Normalization is applied after forming each view. We average probabilities rather than logits:

$$p(y \mid x) = \frac{1}{50} \sum_{v=1}^{50} \mathrm{softmax}(f_{\bar{\theta}}(T_v(x))).$$

The view weights are uniform and are not fitted to hidden test data. The policy preserves a fixed ordering and uses FP32 inference with batch size 64. It increases forward-pass cost while leaving the parameter count unchanged.

Representative alternative recipes

Our initial wide residual network is a custom post-activation design inspired by Wide ResNets (Zagoruyko and Komodakis 2016): nine basic blocks, stage widths 64/128/256, and learned projections at downsampling boundaries. The preactivation alternative uses a 16-channel stem, twelve blocks, and widths 58/116/232. It places BN and ReLU before convolutions and removes the post-addition activation, following the identity-mapping design of He et al. (2016b).

Both wide-network recipes use 200 epochs, batch size 128, cosine-decayed SGD, weight decay 5×10^{-4}, RandAugment, and a mixture of Mixup ($\alpha = 0.2$) and CutMix ($\alpha = 1$; probability 0.5) (Zhang et al. 2018; Yun et al. 2019). For the preactivation network, a paired run replaces SGD updates with sharpness-aware minimization (SAM, $\rho = 0.05$), retaining the remaining configuration (Foret et al. 2021). PyramidNet instead uses the 400-epoch ShakeDrop and Cutout recipe described above. The architecture comparison therefore concerns complete training recipes, while the SGD–SAM pair provides a more focused optimizer comparison.

A separate distillation pair starts from the same 200-epoch preactivation SGD checkpoint and continues for 80 epochs with initial learning rate 0.01 and unmixed augmented images. One branch uses cross-entropy alone; the other equally weights cross-entropy and temperature-scaled KL divergence at temperature 3 (Hinton, Vinyals, and Dean 2015). Its targets average probabilities from two frozen, locally trained teachers: the earlier wide-network EMA and the preactivation SAM EMA. Both teachers use the same training split; only the student is deployed.

Results

Architecture and training-strategy comparison

Table 2 summarizes four representative recipes using FP32 inference and a common ten-view policy: the center and four diagonal offsets at two pixels, each with its horizontal flip. All rows use EMA weights. The early wide-network checkpoint was selected by single-view validation accuracy; the other rows use fixed training endpoints.

The wider preactivation model with SGD performs similarly to the smaller post-activation model. In the matched preactivation pair, SAM improves ten-view accuracy by 0.16

Recipe	Params (M)	Epochs	Acc. (%)
Wide ResNet, SGD	4.328	200	97.26
PreAct wide, SGD	4.806	200	97.24
PreAct wide, SAM	4.806	200	97.40
PyramidNet + SD	4.805	400	97.70

Table 2: Historical recipe comparison on the common 45k/5k split, using EMA and ten-view probability averaging (SD: ShakeDrop). The Wide ResNet row uses epoch 199 (best EMA from a 200-epoch run); the others use the final epoch. Exact parameter counts are 4,327,754, 4,805,542, and 4,804,512 for the wide, preactivation, and pyramidal architectures, respectively.

Weights	Views	Accuracy (%)	NLL
Base EMA400	1	97.48	0.08292
Base EMA400	10	97.70	0.07459
Branch A Raw	10	97.86	0.07267
Branch B Raw	10	97.82	0.07307
Two-branch average	10	97.86	0.07264
Two-branch average	18	97.90	0.07194
Two-branch average	50	97.90	0.07172

Table 3: Measured development results. All models use the same 45,000 training images and 4,804,512 parameters. The 18-view policy uses offsets $\{-2, 0, 2\}^2$ and flips.

percentage points, but its single-view endpoint gain is only 0.02 points (96.84% versus 96.82%). Its recorded training time is 2.57 hours versus 1.37 hours for SGD, a 1.88-fold increase. Under the seven perturbations described below, mean ten-view accuracy is also slightly lower for SAM (93.51% versus 93.78%). Thus its clean-accuracy gain comes with additional computation and mixed robustness results.

PyramidNet reaches the highest accuracy in this comparison, exceeding the preactivation SGD and SAM recipes by 0.46 and 0.30 points. Its mean perturbed accuracy is 94.49%, compared with 93.69% for the earlier wide network. It uses approximately the same parameter budget as the preactivation model but takes 7.26 hours for base training. This combination of stronger clean and perturbed performance motivated spending the remaining computation on PyramidNet refinement rather than further widening the network.

The matched distillation experiment yields a smaller return. At the fixed 280-epoch Raw endpoints, distillation improves single-view accuracy from 96.88% to 97.08%, and ten-view accuracy from 97.28% to 97.34%. The latter change comprises 35 corrected and 32 newly incorrect predictions. Ten-view NLL rises from 0.09446 to 0.14032, while continuation time increases from 30.8 to 41.7 minutes. The small accuracy gain and higher NLL made this recipe a lower priority than the PyramidNet results.

Final-model refinement

Table 3 compares checkpoints and inference policies on the same validation examples. Raw denotes the live trained weights, while EMA denotes their moving average. All continuation endpoints are fixed at epoch 20.

The base EMA model gains 0.22 percentage points from

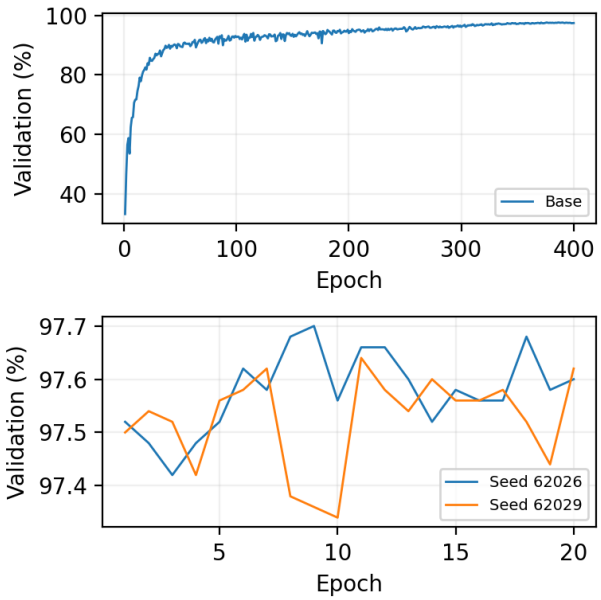


Figure 1: Recorded validation trajectories for the shared base and two continuation branches. Separate axes avoid presenting the branches as one continuous trajectory.

Final-model measure	Value
Trainable parameters	4,804,512
Training images	45,000
Development accuracy	97.90%
Official final test accuracy	87.60%

Table 4: Final submitted model. Test accuracy is measured on the competition's private evaluation subset.

ten-view inference. Continuation branch A adds a further 0.16 points under the same policy, while branch B reaches a similar result. Equal weight averaging preserves branch A's accuracy but does not improve it under ten views. With 50 views, the averaged model correctly classifies 4,895 of 5,000 validation images. Relative to its ten-view prediction, five examples become correct and three become incorrect. The 18-view and 50-view policies have equal accuracy; their disagreements exchange three correct predictions in each direction. The main accuracy improvement comes from continuation; denser views primarily refine the predicted probabilities.

Robustness diagnostics and final evaluation

The existing diagnostic suite applies seven controlled transformations to validation images: brightness scaling by 0.7 and 1.3, contrast scaling by 0.6, grayscale conversion, Gaussian noise with standard deviation 0.05, Gaussian blur with sigma 0.7, and a two-pixel translation. For the averaged model, mean accuracy across these conditions is 94.6371% with ten views, 94.7057% with 18 views, and 94.7543% with 50 views.

The submitted prediction file contains exactly 2,000 rows in official ID order. Its stored checkpoint and CSV hashes allow independent reproduction checks. The original export took approximately 64 seconds for the 2,000 images on the project GPU, excluding image loading and model setup. The shared base and two branches required approximately 7.86 hours of recorded training-workflow time, excluding earlier exploratory experiments. These measurements are hardware- and implementation-specific.

Discussion and Limitations

The experiments favor a deeper, gradually widened residual network over the tested wide-network recipes at a similar parameter budget. This gain requires more computation: PyramidNet doubles the base-training schedule and has more sequential blocks. Short continuation then improves accuracy at relatively low additional training cost. Weight averaging combines nearby endpoints in one network, while dense TTA trades additional forward passes for a small reduction in NLL. The 18-view and 50-view policies have the same clean accuracy, making their inference-cost difference relevant when deployment speed matters.

The validation split was reused for model and inference selection, and most historical recipes have only one training seed. Small differences should therefore be interpreted as development observations rather than precise estimates of unseen-data gains. The final test result is substantially below validation accuracy; its cause remains unresolved. Controlled perturbations broaden the evaluation but do not replace an independent test set. The two final branches also share their base and training data, so they measure variation in continuation rather than independent full-training runs.

The archived checkpoint and fixed inference configuration reproduce the submitted predictions exactly. Fresh CUDA training can vary numerically, so retraining is expected to reproduce the method rather than a bitwise-identical model.

Conclusion

PyramidNet with ShakeDrop provided the strongest clean and perturbed validation performance among the representative recipes examined. SAM offered a smaller accuracy gain at nearly twice the training cost of its SGD counterpart, while distillation produced only a modest improvement in its matched continuation experiment. For the final model, short continuation delivered the clearest incremental gain; fixed weight averaging and denser TTA refined the predictions while retaining a single network within the parameter budget. The released implementation, executed notebook, and final checkpoint support reproduction of this complete pipeline.

Implementation and Tool Disclosure

The implementation uses PyTorch. RandAugment operations are adapted from torchvision with its BSD-3-Clause notice retained. Codex assisted with implementation, debugging, experiment execution, analysis, and manuscript preparation. Sheng Tao is the sole author and is responsible for reviewing the methods and results. No external pretrained weights or training data were used. Detailed sources and reproduction instructions are included in the repository.

References

Cubuk, E. D.; Zoph, B.; Shlens, J.; and Le, Q. V. 2020. RandAugment: Practical Automated Data Augmentation with a Reduced Search Space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*.

DeVries, T.; and Taylor, G. W. 2017. Improved Regularization of Convolutional Neural Networks with Cutout. *arXiv preprint arXiv:1708.04552*.

Foret, P.; Kleiner, A.; Mobahi, H.; and Neyshabur, B. 2021. Sharpness-Aware Minimization for Efficiently Improving Generalization. In *International Conference on Learning Representations*.

Han, D.; Kim, J.; and Kim, J. 2017. Deep Pyramidal Residual Networks. In *CVPR*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016a. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016b. Identity Mappings in Deep Residual Networks. In *European Conference on Computer Vision*.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Izmailov, P.; Podoprikhin, D.; Garipov, T.; Vetrov, D.; and Wilson, A. G. 2018. Averaging Weights Leads to Wider Optima and Better Generalization. In *UAI*.

Yamada, Y.; Iwamura, M.; Akiba, T.; and Kise, K. 2019. ShakeDrop Regularization for Deep Residual Learning. *IEEE Access*, 7: 186126–186136.

Yun, S.; Han, D.; Oh, S. J.; Chun, S.; Choe, J.; and Yoo, Y. 2019. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.

Zagoruyko, S.; and Komodakis, N. 2016. Wide Residual Networks. In *British Machine Vision Conference*.

Zhang, H.; Cisse, M.; Dauphin, Y. N.; and Lopez-Paz, D. 2018. mixup: Beyond Empirical Risk Minimization. In *International Conference on Learning Representations*.